
Two Ways to Combine Numerical and Classification Objectives in Time-Series Foundation Models

Yewei Huang
istina@sjtu.edu.cn

Abstract

Transformer-based time-series foundation models inherit the expressive sequence modeling structure of large language models, but their loss functions must still handle the ordered and continuous nature of forecasting targets: some models predict numerical values directly, while others discretize values and formulate forecasting as classification, and both choices have distinct advantages and limitations. We study this problem through two lightweight adaptations: changing the fine-tuning objective and replacing the prediction head to switch between continuous and discretized output formulations. Experiments on pretrained Chronos and TimesFM models show that classification-style and distance-aware objectives can each help, but their effect depends strongly on the base architecture and evaluation metric.

1 Introduction

Time series forecasting is a core tool for planning under uncertainty. It is used when decisions must be made before future demand, traffic, weather, prices, or resource consumption are observed. Classical forecasting methods were built around explicit statistical structure, including autoregressive integrated moving-average models and exponential smoothing [Box and Jenkins, 1970, Winters, 1960]. They remain useful as transparent baselines. However, modern applications often require a single model to adapt across many related series, frequencies, and domains, which has pushed forecasting toward global neural models and pretrained representations.

Transformers accelerated this shift [Vaswani et al., 2017]. Earlier Transformer-based forecasting models adapted attention to long temporal contexts through efficiency, decomposition, or patching mechanisms [Zhou et al., 2021, Wu et al., 2021, Nie et al., 2023]. More recently, forecasting foundation models pretrain on broad time-series corpora and transfer to new datasets with little task-specific data. TimesFM follows this direction with a decoder-only architecture for continuous forecasting [Das et al., 2024]. Chronos takes a more language-model-like route: continuous observations are scaled, quantized into a finite vocabulary, and then modeled with Transformer language-model architectures under cross-entropy [Ansari et al., 2024].

These two representative models reveal a basic design tension in time-series foundation models. Forecast values are continuous and ordered, so losses such as mean-squared error, Huber loss, quantile loss, and continuous ranked probability score naturally reflect numerical distance or distributional calibration [Huber, 1964, Koenker and Bassett, 1978, Matheson and Winkler, 1976]. However, purely numerical objectives can also encourage conservative forecasts: when future trajectories are uncertain or multi-modal, optimizing an average error may push the model toward a safe central value that is numerically plausible but not very informative. At the same time, discretization turns forecasting into classification, making it compatible with the stable probabilistic training objectives used by language models. This can represent uncertainty more naturally over bins, but plain cross-entropy treats all wrong classes equally: a token one bin away and a token thousands of bins away receive the same categorical penalty structure. Recent work therefore tries to inject ordinal or distance-aware information into classification-style forecasting objectives, including Wasserstein-based fine-tuning [Villani, 2008, Chernov, 2024] and ordinal cross-entropy [Wang et al., 2025].

This project studies the same objective-design problem from two practical angles. First, we keep the pretrained architecture fixed and change the fine-tuning loss. For Chronos, whose prediction space is already discretized, this allows us to compare bin-index regression, Wasserstein-style distances, CRPS-like objectives, ordinal cross-entropy, and mixtures of cross-entropy and distance-aware losses without changing the model architecture. Second, we modify the prediction head of a pretrained forecasting model. For TimesFM, which originally predicts continuous values, this requires adding or replacing a head so that the same backbone can be optimized with a classification-style objective. This route is more intrusive than changing the loss alone, because it changes the output interface of the pretrained model and can alter the fine-tuning dynamics.

Our experiments compare these choices on a held-out Monash forecasting dataset under zero-shot and parameter-efficient fine-tuning settings. The results do not support a single universally best objective. Different losses show different strengths during fine-tuning, suggesting that objective design remains an important adaptation choice for pretrained forecasting models. In addition, the head-replacement approach can achieve competitive results after fine-tuning, which indicates that changing the output formulation is also a promising way to combine classification-style and distance-aware forecasting. Overall, the evidence suggests that these objectives are complementary, but their benefit depends on how they match the pretrained architecture, output representation, and evaluation metric.

2 Related Work

2.1 Time Series Forecasting

Given a historical time series $x_{1:T} = (x_1, \dots, x_T)$, time series forecasting aims to predict the next H values $x_{T+1:T+H}$. A point forecasting model learns a function

$$\hat{x}_{T+1:T+H} = f_{\theta}(x_{1:T}), \quad (1)$$

while a probabilistic forecasting model estimates a predictive distribution

$$p_{\theta}(x_{T+1:T+H} \mid x_{1:T}). \quad (2)$$

In practice, this distribution is often represented by samples, quantiles, or a discretized probability mass over bins. Classical methods such as ARIMA and exponential smoothing model this problem with explicit assumptions about autoregression, trend, and seasonality [Box and Jenkins, 1970, Winters, 1960]. Recent neural methods instead learn global representations across many series, and Transformer-based models have become a common architecture for long-context forecasting [Vaswani et al., 2017, Zhou et al., 2021, Wu et al., 2021, Nie et al., 2023].

2.2 Chronos

Chronos is a time-series foundation model that reformulates forecasting as a language modeling problem [Ansari et al., 2024]. Instead of predicting raw real values directly, it first scales each time series and maps the scaled values into a fixed vocabulary of discrete bins. This makes Chronos compatible with existing Transformer language-model architectures and the standard cross-entropy objective. Forecasts are produced by autoregressively sampling future tokens and mapping them back to numerical bin centers. This formulation gives Chronos a clean probabilistic output over possible future values, but it also inherits a limitation of categorical training: plain cross-entropy rewards the correct bin without considering the numerical distance between nearby and far-away mistakes. This makes Chronos a useful testbed for distance-aware fine-tuning losses.

2.3 TimesFM

TimesFM takes a different route: it is a decoder-only forecasting foundation model that predicts future values in a continuous output space [Das et al., 2024]. The model uses patching to compress long time-series contexts into a shorter sequence of input units, then decodes future patches. Instead of producing a categorical distribution over token bins, TimesFM outputs real-valued forecasts for future time steps in a batch. Its output head therefore keeps predictions in the original numerical space instead of passing through a discrete vocabulary. This preserves the numerical geometry of forecasting targets, but continuous objectives can also bias the model toward safe central predictions when the future is uncertain. TimesFM is therefore useful for the opposite experiment from Chronos:

rather than adding distance awareness to a classification model, we study whether a continuous forecasting backbone can benefit from an added or replaced classification-style head.

2.4 Forecasting Objectives

The choice of objective determines what kind of forecast the model is encouraged to produce. Mean-squared error and Huber loss are numerical objectives that penalize pointwise forecast errors, with Huber loss reducing the influence of large deviations [Huber, 1964]. Quantile losses optimize asymmetric errors for specific quantile levels [Koenker and Bassett, 1978], while CRPS evaluates an entire predictive distribution [Matheson and Winkler, 1976]. Classification objectives such as cross-entropy optimize the probability of the correct bin, but ignore numerical distance unless the ordinal structure is added explicitly. Recent Wasserstein and ordinal-cross-entropy losses are examples of this distance-aware direction [Villani, 2008, Chernov, 2024, Wang et al., 2025]. In our experiments, these objective families mainly motivate the loss-level fine-tuning study, while the head-replacement study focuses on switching between the original continuous objective and cross-entropy over discretized targets. We give the concrete definitions of the losses used in our experiments in Appendix A.

3 Method

3.1 Overview

We study two ways to combine classification-style and distance-aware objectives on pretrained time-series models [Ansari et al., 2024, Das et al., 2024]. The first route keeps the pretrained model architecture fixed and only changes the fine-tuning objective. This is applied to Chronos, whose output space is already discretized. The second route changes the prediction head of a continuous forecasting model. This is applied to TimesFM, where we compare the original continuous head with a newly introduced classification head.

In both routes, the backbone is pretrained; we only change the fine-tuning loss or the output head before evaluating on held-out series.

3.2 Loss-Level Fine-Tuning for Chronos

Chronos converts a real-valued time series into discrete token bins and predicts a distribution over the bin vocabulary [Ansari et al., 2024]. Let K denote the number of numerical bins, $b_t \in \{0, \dots, K-1\}$ the target bin index at a valid prediction position, and $p_{t,k}$ the model probability assigned to bin k . The original Chronos objective is token cross-entropy,

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \log p_{t,b_t}, \quad (3)$$

where $\mathcal{M}$ is the set of non-padding target positions.

To inject numerical distance into this discrete output space, we compute losses in the bin-index space. The soft prediction is the expected bin index

$$\bar{b}_t = \sum_{k=0}^{K-1} k p_{t,k}. \quad (4)$$

The bin-index MSE loss is then

$$\mathcal{L}_{\text{bin-MSE}} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \left(\frac{\bar{b}_t - b_t}{K-1} \right)^2. \quad (5)$$

The normalization by $(K-1)^2$ keeps this loss on a comparable scale to cross-entropy. We also test a mixed objective that adds a λ -weighted CE term to the normalized bin-index MSE.

Beyond MSE, we evaluate several distance-aware losses on the same bin distribution. For Wasserstein-style losses, the target is treated as a point mass at b_t [Villani, 2008, Chernov, 2024]:

$$\mathcal{L}_{W_p} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \frac{\left(\sum_{k=0}^{K-1} p_{t,k} |k - b_t|^p \right)^{1/p}}{K-1}, \quad p \in \{1, 2\}. \quad (6)$$

We also use a Huber variant in bin-index space [Huber, 1964]. For distributional losses, we compare cumulative distributions. Let $F_t(k) = \sum_{j \leq k} p_{t,j}$ be the predicted CDF and $G_t(k)$ the target CDF. The CRPS-style loss uses squared CDF error [Matheson and Winkler, 1976], while ordinal cross-entropy replaces the squared penalty with binary cross-entropy on the cumulative probabilities [Wang et al., 2025]. All these losses are computed only on valid target positions.

3.3 Head-Level Adaptation for TimesFM

TimesFM originally predicts future values through a continuous forecasting head [Das et al., 2024]. We use this original formulation in two ways: as the zero-shot baseline and as a continuous fine-tuning baseline with LoRA. We also train the original forecasting head from scratch as a control.

To study a classification-style output on TimesFM, we add a new CE head on top of the pretrained backbone. The backbone first maps patched and normalized inputs to output embeddings. The CE head maps each output embedding to K logits for each future step in the output patch:

$$\ell_{h,1:K} = g_\phi(r_h), \quad (7)$$

where r_h is the TimesFM output representation for horizon step h and g_ϕ is the classification head. Targets are discretized with uniform bins over $[-10, 10]$ in the RevIN normalized space. If $\tilde{x}_{T+h}$ is the normalized target value and $q(\cdot)$ maps it to a bin index, the CE-head loss is

$$\mathcal{L}_{\text{TimesFM-CE}} = -\frac{1}{H} \sum_{h=1}^H \log \text{softmax}(\ell_h)_{q(\tilde{x}_{T+h})}. \quad (8)$$

We instantiate this CE head with $K = 64$ and $K = 256$ bins. The 64-bin head is smaller and cheaper to adapt, while the 256-bin head gives a finer discretization but introduces more output parameters. In our implementation, these choices correspond to about 22.6M and 85.6M CE-head parameters, respectively. Because the CE head is newly initialized rather than inherited from the pretrained TimesFM checkpoint, we first pretrain it with the backbone frozen, then use it for downstream fine-tuning. We also vary the scale of this head-pretraining stage: the small pool uses five datasets with 200k sampled windows, the 256-bin pool uses seven datasets with 200k sampled windows, and the large-pool 64-bin run uses a larger 300k-window generic pool.

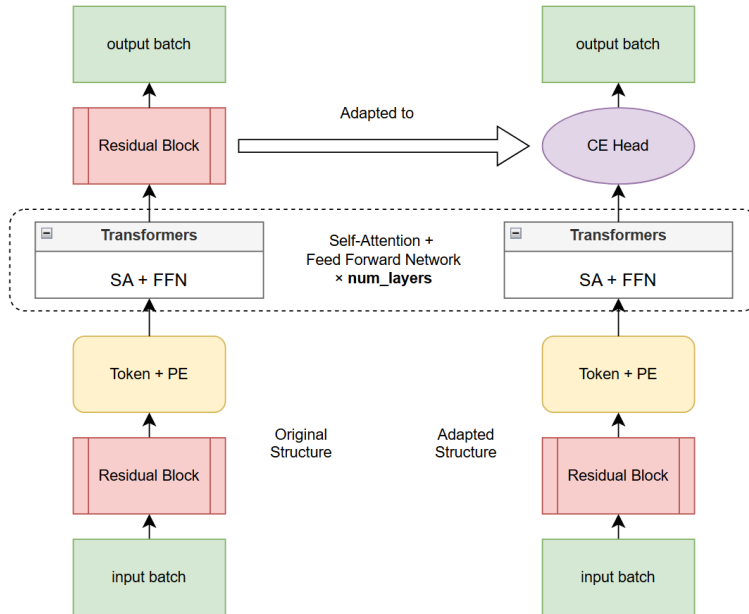


Figure 1: Head-level adaptation for TimesFM. The original model uses a continuous forecasting head, while our adapted version replaces this output head with a CE head over discretized targets.

At inference time, the CE head produces a probability distribution over bins. We convert this distribution back to numerical forecasts by taking the softmax-weighted expectation of the bin centers in normalized space and then applying the inverse RevIN transform. Quantile forecasts for WQL evaluation are obtained by inverting the predicted CDF over bins. Thus the CE head changes the training target from continuous regression to discretized classification, but the final outputs are still converted back to real-valued forecasts for evaluation.

3.4 Parameter-Efficient Fine-Tuning

All main downstream experiments use parameter-efficient fine-tuning rather than full-model training, following the low-rank adapter idea in LoRA [Hu et al., 2021]. For Chronos, LoRA adapters are attached to the pretrained T5 backbone while the loss function is varied. For TimesFM, LoRA is applied to the backbone during downstream adaptation, and the output head is trained with a separate learning rate. This separation is important for the CE-head setting: the newly introduced head can adapt much faster than the pretrained backbone, and using the same aggressive learning rate for both can easily destabilize fine-tuning.

These two settings cover both directions of the classification-numerical design choice: adding numerical distance information to a classification model, and adding a classification head to a numerical forecasting model.

4 Experiments

Unless otherwise stated, all reported runs use seed 42. WQL is computed from quantile pinball losses, and MASE is a scale-normalized absolute point-forecast error; lower values are better for both. Because Chronos and TimesFM use different split and evaluation pipelines, their absolute scores should be read within each model family rather than as strict cross-model rankings.

4.1 Chronos Loss Fine-Tuning

We first evaluate the loss-level adaptation route on Chronos. The base model is `amazon/chronos-t5-base`. We use the `tourism_monthly` dataset from the Monash forecasting archive [Godahewa et al., 2021], with a sequence-level split of 293 training series and 73 held-out evaluation series. This dataset is used because it is not part of the listed in-domain Monash evaluation set for Chronos. The prediction length is 24, and quantile evaluation uses 20 generated samples per series. All fine-tuned Chronos variants use LoRA adapters; only the fine-tuning objective is changed. We report weighted quantile loss (WQL) and MASE.

Table 1: Chronos fine-tuning results on `tourism_monthly`.

Model	WQL	MASE
Zero-shot baseline	1.5441	1.6617
CE+LoRA	1.2244	1.3806
bin-MSE+LoRA	1.2651	1.4176
bin-MSE+CE+LoRA	1.2638	1.4147
bin-W1+LoRA	1.2750	1.3151
bin-W2+LoRA	1.3252	1.3352
bin-CRPS+LoRA	1.2325	1.3713
bin-OrdinalCE+LoRA	1.2357	1.3765
bin-Huber+LoRA	1.2562	1.3803

Table 1 and Figures 2 and 3 summarize the Chronos results. The zero-shot baseline records WQL 1.5441 and MASE 1.6617. All LoRA fine-tuning variants reduce both metrics. CE+LoRA obtains the lowest WQL, 1.2244. bin-W1+LoRA obtains the lowest MASE, 1.3151. bin-CRPS+LoRA and bin-OrdinalCE+LoRA are the next closest variants to CE+LoRA in WQL. bin-MSE+CE+LoRA remains close to bin-MSE+LoRA in both metrics. In the scatter plot, the zero-shot result is shown as an off-scale reference, and the remaining points compare the fine-tuned objectives.

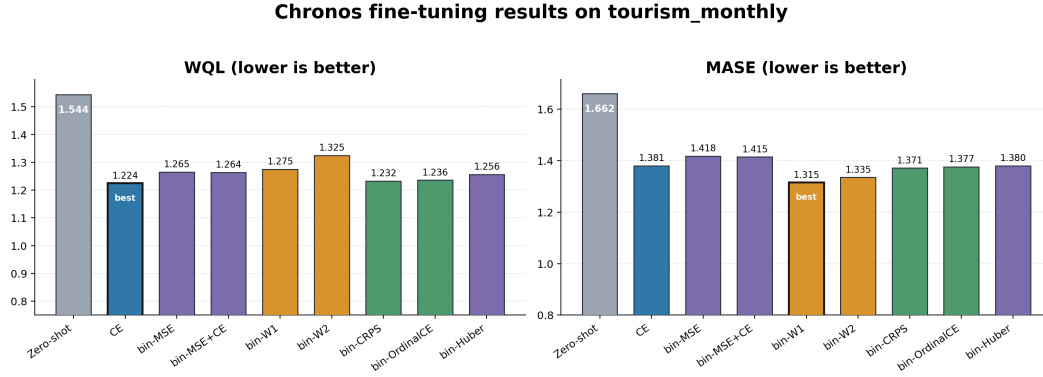


Figure 2: Chronos WQL and MASE comparisons. The zero-shot baseline is included as the first bar, while the fine-tuned variants form the main comparison. CE+LoRA obtains the best WQL, while bin-W1+LoRA obtains the best MASE.

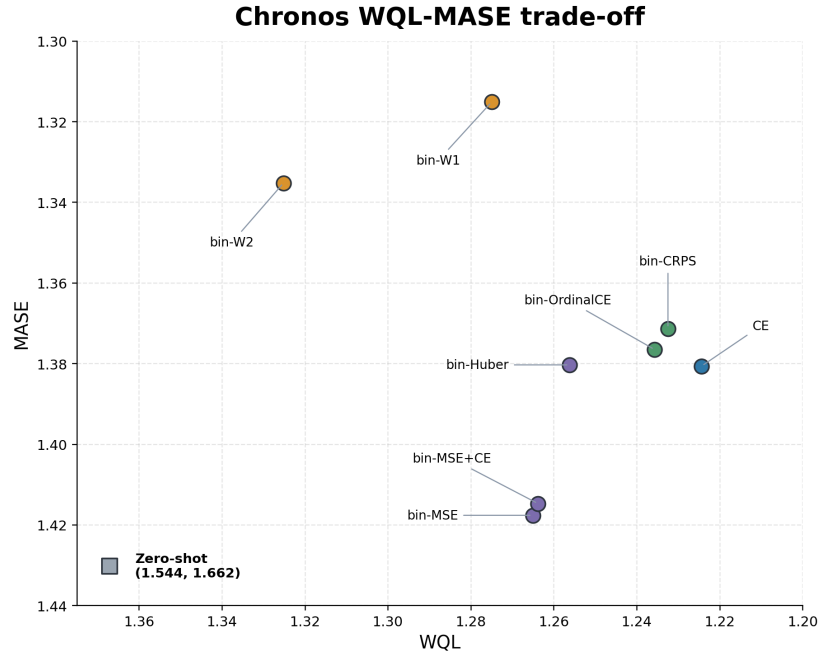


Figure 3: Chronos WQL-MASE trade-off. Different objectives occupy different regions of the metric space, showing that the best probabilistic objective is not necessarily the best point forecasting objective. The zero-shot baseline is shown as an off-scale reference.

4.2 TimesFM Head Adaptation

We next evaluate the head-level adaptation route on TimesFM. The backbone is `google/timesfm-2.5-200m-pytorch`. We do not reproduce the original Google pretraining of this backbone; instead, our additional pretraining only trains a replacement output head on selected Monash datasets while keeping the backbone frozen. The downstream target is again `tourism_monthly`, but TimesFM uses a different split from Chronos: 257 training series, 36 validation series, and 73 test series. The context length is 128 and the prediction length is 24. To avoid target leakage, `tourism_monthly` is excluded from all head-pretraining pools.

Table 2 summarizes the data used for our replacement-head pretraining. The small pool was used for the scratch original forecast head and the 64-bin CE head. The 256-bin CE head uses the same pool with two additional monthly datasets. We also tested a 64-bin CE head pretrained with a larger generic pool. All these datasets come from the Monash forecasting archive [Godaheva et al., 2021].

Table 2: TimesFM replacement-head pretraining data. The TimesFM backbone itself is loaded from the public pretrained checkpoint; this table only describes the additional head-pretraining data used in this project.

Setting	Datasets	Use
small pool	<code>m4_monthly</code> , <code>m4_daily</code> , <code>weather</code> , <code>electricity_hourly</code> , <code>traffic_hourly</code>	original head, CE64
CE256 pool	small pool + <code>m3_monthly</code> , <code>m1_monthly</code>	CE256
large pool	starter: <code>m4_monthly</code> , <code>temperature_rain</code> , <code>weather</code> , <code>traffic_hourly</code> , <code>electricity_hourly</code> ; large: <code>kaggle_web_traffic</code> , <code>rideshare</code> , <code>vehicle_trips</code> , <code>kdd_cup_2018</code> , <code>covid_deaths</code> , <code>nm5_daily</code> , <code>fred_md</code> , and the overlapping M4, weather, traffic, and electricity datasets	CE64 large-pool

For head pretraining, we draw random training windows from the selected datasets, use a sequence-level validation/test split, and train only the output head. The original-head and CE64 runs use 200k random windows, 5 epochs, and a head learning rate of 10^{-3} . The CE256 run also uses 200k windows and 5 epochs, but a smaller batch size because the 256-bin head has many more parameters. Downstream LoRA fine-tuning on `tourism_monthly` then updates the backbone adapters and, unless otherwise stated, also updates the head with a separate learning rate.

Table 3: TimesFM results on `tourism_monthly`. “Direct” means that the pretrained replacement head is evaluated without downstream fine-tuning; “LoRA” means downstream fine-tuning on the `tourism_monthly` training split. For CE-head rows, `head_lr` is the output-head learning rate and `lora_lr` is the backbone-adapter learning rate. Lower is better.

Model	Setting	WQL	MASE
Base zero-shot	original TimesFM head	1.1899	1.1688
Base LoRA	original regression loss + LoRA, <code>lr=1e-4</code>	1.2058	1.1410
Original head direct	scratch original head, small-pool pretrain	1.2795	1.2389
Original head + LoRA	small-pool original head + LoRA, <code>lr=1e-4</code>	1.2049	1.1468
CE64 direct	64-bin CE head, small-pool pretrain	1.4605	1.3809
CE64 + LoRA	<code>head_lr=5e-5</code> , <code>lora_lr=5e-5</code>	1.2308	1.1162
CE256 direct	256-bin CE head	1.3624	1.2862
CE256 + LoRA	<code>head_lr=1e-5</code> , <code>lora_lr=5e-5</code>	1.2691	1.2336
CE64 large-pool direct	64-bin CE head, large-pool pretrain	4.5760	5.5845
CE64 large-pool + LoRA	<code>head_lr=5e-5</code> , <code>lora_lr=5e-5</code>	1.4641	1.3968

Table 3 summarizes the TimesFM results. The base zero-shot model obtains the lowest WQL, 1.1899. Base LoRA changes the metrics to WQL 1.2058 and MASE 1.1410. The scratch original head obtains WQL 1.2795 and MASE 1.2389 before downstream LoRA, then WQL 1.2049 and MASE 1.1468 after LoRA. For the classification heads, CE64 direct records WQL 1.4605 and MASE 1.3809, while CE64+LoRA records the lowest MASE, 1.1162. CE256+LoRA with `head_lr=1e-5` improves over CE256 direct, changing the metrics from WQL 1.3624 and MASE 1.2862 to WQL 1.2691 and MASE 1.2336. The CE64 large-pool head records WQL 4.5760 and MASE 5.5845 before downstream LoRA, then WQL 1.4641 and MASE 1.3968 after downstream LoRA.

5 Conclusion

This project studies how loss functions and output formulations affect pretrained time-series forecasting models. The Chronos experiments show that classification-style training and numerical-distance objectives emphasize different forecasting behavior. The original CE objective remains strongest for WQL, while distance-aware objectives such as bin-W1, CRPS-style loss, and ordinal CE improve or balance other aspects of the forecast. This suggests that the two objective families are not simply substitutes: categorical probability modeling and numerical distance information provide complementary signals during fine-tuning.

The TimesFM experiments point in the opposite direction and lead to the same broader conclusion. Starting from a numerical forecasting backbone, adding a CE head makes a classification-style adaptation feasible, and CE64 with LoRA achieves the best MASE among our TimesFM variants. This result is important because the benefit is not limited to probabilistic bin prediction; the classification formulation also helps point-to-point forecasting after fine-tuning. In other words, combining the strengths of classification objectives and numerical forecasting backbones can expose capabilities that are not visible under the original output formulation alone.

Overall, the results support the view that forecasting foundation models should not be tied too rigidly to either pure regression or pure classification objectives. A model can benefit from probability mass over discrete outcomes, from distance-aware numerical structure, or from an architecture that connects the two. At the same time, this project is limited by computation and time: we run the complete pipeline on a single target dataset, so future work should verify whether the same trends hold across larger and more diverse benchmarks. The TimesFM results also show that head pretraining data matters. The small-pool CE64 head becomes strong after downstream LoRA, but the larger-pool CE64 head transfers poorly before fine-tuning and remains weak after LoRA. This suggests that blindly stacking heterogeneous time-series datasets for head pretraining can cause severe negative transfer rather than stable improvement. Since our CE heads are trained only on selected Monash datasets rather than on a corpus comparable to the original TimesFM pretraining data, stronger and better matched CE-head pretraining remains an important direction for future work.

Acknowledgements

Thanks to AKrry5307 for the inspiration and encouragement throughout this project.

Thanks to Codex+GPT, Qoder+Qwen, and Kilo Code+DeepSeek for surviving the debugging, plotting, and paper-polishing loop with me.

References

- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.
- George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970.
- Andrei Chernov. Fine-tuning a time series foundation model with wasserstein loss. *arXiv preprint arXiv:2409.15367*, 2024.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. *arXiv preprint arXiv:2310.10688*, 2024.
- Rakshitha W. Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

- Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35 (1):73–101, 1964.
- Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978.
- James E. Matheson and Robert L. Winkler. Scoring rules for continuous probability distributions. *Management Science*, 22(10):1087–1096, 1976.
- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Cedric Villani. *Optimal Transport: Old and New*. Springer, 2008.
- Jieting Wang, Huimei Shi, Feijiang Li, and Xiaolei Shang. Beyond MSE: Ordinal cross-entropy for probabilistic time series forecasting. *arXiv preprint arXiv:2511.10200*, 2025.
- Peter R. Winters. Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3):324–342, 1960.
- Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11106–11115, 2021.

A Loss Definitions

This appendix collects the objective definitions referenced in the main text. Let $\mathcal{M}$ be the set of valid forecast positions, K the number of bins, b_t the target bin index, and $p_{t,k}$ the predicted probability of bin k at position t . The standard bin cross-entropy is

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \log p_{t,b_t}. \quad (9)$$

For losses that use a soft numerical prediction in bin space, we use the expected bin index $\bar{b}_t = \sum_{k=0}^{K-1} k p_{t,k}$. The normalized bin-index MSE is

$$\mathcal{L}_{\text{bin-MSE}} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \left(\frac{\bar{b}_t - b_t}{K-1} \right)^2. \quad (10)$$

The mixed Chronos variant adds a λ -weighted CE term to this normalized bin-index MSE.

For Wasserstein-style losses, the target distribution is a point mass at the target bin:

$$\mathcal{L}_{W_p} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \frac{\left(\sum_{k=0}^{K-1} p_{t,k} |k - b_t|^p \right)^{1/p}}{K-1}, \quad p \in \{1, 2\}. \quad (11)$$

The Huber variant applies the standard Huber penalty to the bin-index error $e_t = \bar{b}_t - b_t$:

$$\rho_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \leq \delta, \\ \delta \left(|e| - \frac{1}{2}\delta \right), & |e| > \delta. \end{cases} \quad (12)$$

For CDF-based losses, define the predicted CDF $F_t(k) = \sum_{j \leq k} p_{t,j}$ and the target CDF $G_t(k) = \mathbf{1}[k \geq b_t]$. The CRPS-style loss is

$$\mathcal{L}_{\text{CRPS}} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \frac{1}{K} \sum_{k=0}^{K-1} (F_t(k) - G_t(k))^2. \quad (13)$$

Ordinal cross-entropy uses binary cross-entropy on the same cumulative probabilities:

$$\mathcal{L}_{\text{OrdCE}} = -\frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \frac{1}{K} \sum_{k=0}^{K-1} [G_t(k) \log F_t(k) + (1 - G_t(k)) \log(1 - F_t(k))]. \quad (14)$$

For TimesFM CE-head experiments, the continuous normalized target $\tilde{x}_{T+h}$ is first mapped to a bin $q(\tilde{x}_{T+h})$, and the head is trained with the same cross-entropy form over the horizon steps. Quantile loss, used in WQL evaluation, is the pinball loss

$$\rho_\tau(y, \hat{y}) = \max\{\tau(y - \hat{y}), (\tau - 1)(y - \hat{y})\}, \quad (15)$$

aggregated over the evaluated quantile levels and normalized by the scale of the target series.